

TESPEC: Proof-Backed Instance Testing of C Heap Specifications

Anonymous Authors

Abstract

Formal specifications are increasingly generated by automated tools and large language models, yet checking whether a generated specification captures the intended behavior remains difficult. Runtime checking and property-based testing now support substantial fragments of separation logic, including complex C heaps, but rely on contracts with a supplied computational interpretation. At the same time, specification-evaluation systems can prove declarative propositions on concrete value-level examples. Our target lies between these lines: existing QCP contracts that combine mutable C heaps with ghost values, quantifiers, and application-specific Rocq propositions for which no runtime monitor has been supplied.

We present TESPEC, a proof-backed specification-testing tool for QCP-annotated C functions. Its input is a C implementation and contract together with typed bindings for C parameters and logical `With` variables. Instead of compiling a second executable oracle, TESPEC follows the concrete program path in a modified QCP symbolic executor, directly executes finite loops and visible calls, and specializes the original pre/postcondition with the observed heap transition. QCP and SMT discharge routine obligations; unresolved formulas remain visible Rocq goals that may be completed by a human or proof agent. The deterministic core, rather than the agent, checks all inputs and evidence.

Through a running minimum-subarray example, we show how one test simultaneously fixes C arguments, a concrete array heap, and its logical list view, and how a proof-oriented postcondition becomes a closed proposition about the observed return and final heap. TESPEC is not a replacement for universal verification: it provides reproducible evidence about selected concrete heap behaviors without requiring the original contract to be rewritten as a runtime checker.

1 Introduction

Formal verification establishes that an implementation satisfies a specification, but does not establish that the specification expresses what a developer intended. This familiar oracle problem has become more visible as large language models (LLMs) are used to generate preconditions, postconditions, invariants, and auxiliary predicates. A generated contract can parse, type-check, and even verify while remaining vacuous, incomplete, or semantically misaligned.

Specification evaluation has therefore become an independent software engineering problem. FormalBench evaluates inferred specifications using consistency and completeness checks [Le-Cong et al., 2025]. CLEVER uses formal equivalence to curated references [Thakur et al., 2025]. VERINA separates code, specification, and proof generation and uses positive and negative examples to evaluate specification quality [Ye et al., 2026]. COINS specializes declarative Rocq propositions to concrete input/output pairs and asks a proof checker to validate the resulting instances [Yang et al., 2026]. These approaches avoid relying solely on whether one implementation verifies, but their examples are predominantly relations over ordinary values.

Property-based testing (PBT) attacks a closely related problem from the testing side. QuickCheck repeatedly evaluates an executable property on generated inputs; QuickChick brings this workflow to Coq, and FuzzChick improves input exploration with coverage guidance [Claessen and Hughes, 2000, Paraskevopoulou et al., 2015, Lampropoulos et al., 2019]. For heap-manipulating C, Fulminate instruments CN contracts with runtime ownership checks, while Bennet synthesizes randomized heap generators from the restricted CN separation-logic fragment [Banerjee et al., 2025, Amer and Pierce, 2025]. These systems demonstrate that heap-aware specification testing is both feasible and useful. Their runtime approach deliberately relies on a specification fragment with a predictable computational interpretation.

QCP occupies a different point in the design space. Its contracts may use separating conjunction, recursive predicates, quantifiers, arbitrary `With` types, old-state values, and imported Rocq definitions [Wu et al., 2026]. This expressiveness is useful for verification but does not by itself provide a runtime interpretation. Some QCP spatial predicates, such as arrays and input-directed lists, clearly admit monitors; others invoke application-specific Rocq propositions that would first have to be translated into decision procedures. Doing so manually duplicates the oracle and creates a new equivalence obligation between the monitor and the contract.

This paper explores a pragmatic alternative:

Can we obtain useful test feedback for proof-oriented C heap contracts without first rewriting them as executable properties?

We implement this idea in TESPEC. A test case supplies concrete C inputs, heap data, and typed values for the contract’s logical variables. A modified QCP executor follows the selected execution, unrolling finite loops and entering visible callees instead of asking for loop invariants and callee contracts. At function return, the tool substitutes the initial and final states into the original **Require/Ensure**, simplifies closed operations, and invokes the ordinary QCP/Rocq proof pipeline. A proof-backed success is conclusive for that instance; an unresolved goal is reported rather than silently accepted.

The approach resembles PBT in its use of concrete cases and fast feedback, but differs in its oracle: the property need not arrive with an executable decision procedure. It resembles instance-based specification evaluation, but the observed behavior is a mutable heap transition rather than an immutable pair. It complements Fulminate and Bennet by trading randomized runtime throughput for a wider assertion language and proof-producing residual path.

This paper makes the following software-engineering contributions:

1. We formulate testing a QCP contract as proof checking over an indexed concrete heap transition, preserving the original assertion as the oracle.
2. We introduce a practical interface for supplying typed C inputs, heap descriptions, and logical-variable bindings without encoding expected outputs in an executable test harness.
3. We adapt QCP symbolic execution to follow finite concrete loops and visible calls without requiring test authors to write invariants or callee specifications for the selected case.
4. We combine generic closed-term simplification, QCP/SMT automation, and residual Rocq proofs in a transparent evidence workflow whose optional agent layer cannot determine verdicts by itself.

We deliberately do not claim that passing finitely many cases proves a contract universally correct. TESPEC is a specification-testing and debugging tool: it supplies machine-checked evidence for selected behaviors and makes unsupported or unresolved cases explicit.

2 Motivating Example

2.1 A proof-oriented contract over a C heap

We begin with a small but representative function adapted from our subject corpus. It reads a heap-resident array and returns the minimum sum of any nonempty contiguous subarray. Its QCP contract connects three views of the same input: the C pointer and length, a logical list `nums_l`, and the bytes owned by the array predicate.

```
1  /* abridged from the artifact case */
2  /*@ Extern Coq
3      (problem_114_spec : list Z -> Z -> Prop)
4      (kadane_int64_range : list Z -> Prop) */
5  long long p114_minSubArraySum(long long *nums, int nums_size)
6  /*@ With (nums_l : list Z)
7      Require
8          1 <= nums_size &&
9          nums_size == Zlength(nums_l) &&
10         kadane_int64_range(nums_l) &&
11         LongArray::full(nums, nums_size, nums_l)
12      Ensure
13         problem_114_spec(nums_l, __return) &&
14         LongArray::full(nums, nums_size, nums_l)
15  */
16  {
17      long long cur = nums[0], best = nums[0];
18      for (int i = 1; i < nums_size; ++i) {
19          cur = cur < 0 ? cur + nums[i] : nums[i];
20          if (cur < best) best = cur;
21      }
22      return best;
23  }
```

The imported Rocq proposition is intentionally declarative. Omitting minor library syntax, it says

$$\text{MinSpec}(xs, r) \triangleq (\exists pre, seg, suf. seg \neq [] \wedge xs = pre ++ seg ++ suf \wedge \text{sum}(seg) = r) \\ \wedge (\forall pre, seg, suf. seg \neq [] \wedge xs = pre ++ seg ++ suf \Rightarrow r \leq \text{sum}(seg)).$$

The first conjunct requires a witness attaining r ; the second states minimality over all subarrays. This is ordinary proof-oriented Rocq: `problem_114_spec` has type `Prop`, contains nested quantification over lists, and is imported as a logical definition rather than as a Boolean procedure.

The heap is not, by itself, the obstacle. A monitor can walk `LongArray::full(nums, nums_size, nums_1)`, check allocation and ownership, and compare the six cells with `nums_1`. Fulminate gives precisely this kind of computational interpretation to CN resources, and Bennet runs the interpretation in reverse to generate satisfying heaps [Banerjee et al., 2025, Aamer and Pierce, 2025]. The missing component is a monitor for the imported `MinSpec`. It is decidable for a finite list, but the QCP source supplies no decision procedure. A user could write one, but would then have to maintain a second oracle and justify that it is equivalent to the Rocq proposition.

2.2 One indexed heap behavior

A `TESPEC` case makes the pre-state explicit:

```
1 {
2   "args": {"nums": 12288, "nums_size": 6},
3   "values": {"nums_1": [2, 3, 4, 1, 2, 4]}
4 }
```

The binding does not provide an expected return. It states that address 12288 is the root of a six-element array whose logical view is the given list. The spatial precondition materializes the corresponding typed heap. Concrete execution then follows five loop iterations:

step	current element	cur	best
entry		2	2
1	3	3	2
2	4	4	2
3	1	1	1
4	2	2	1
5	4	4	1

The observed behavior is

$$(nums_size = 6, H_0 = [2, 3, 4, 1, 2, 4]) \longrightarrow (r = 1, H_1 = H_0).$$

Substitution eliminates the C locals, loop path, pointer address, return placeholder, and abstract pre-state variable. The terminal contract becomes, schematically,

$$\text{MinSpec}([2, 3, 4, 1, 2, 4], 1) \wedge \text{ArrayEq}(H_1, 12288, [2, 3, 4, 1, 2, 4]).$$

The spatial conjunct is checked against the concrete final heap. The first conjunct is retained as a closed Rocq proposition: simplification may unfold finite list operations, SMT may close arithmetic leaves, and any remaining quantified reasoning is exported as a residual goal. Failure to close that goal yields `UNKNOWN`, not a false counterexample.

2.3 Why both positive and negative behaviors matter

The example also shows why executing a correct implementation is insufficient for evaluating specification quality. Consider a plausible weakened candidate that keeps only the existence conjunct of `MinSpec`. It accepts the observed return 1, but it also accepts the invalid terminal behavior $r = 2$, because the singleton subarray [2] has sum 2. Conversely, an over-strong candidate requiring $r < 0$ rejects the valid behavior above.

candidate contract	valid $r = 1$	invalid $r = 2$
full existence-and-minimality relation	accept	reject
existence only	accept	accept
full relation plus $r < 0$	reject	reject

Positive heap executions therefore expose over-restriction, while frozen negative terminal behaviors expose under-specification. A strict semantic rejection must be backed by a checked proof of the instantiated proposition's negation; an execution error or failed proof search is not enough.

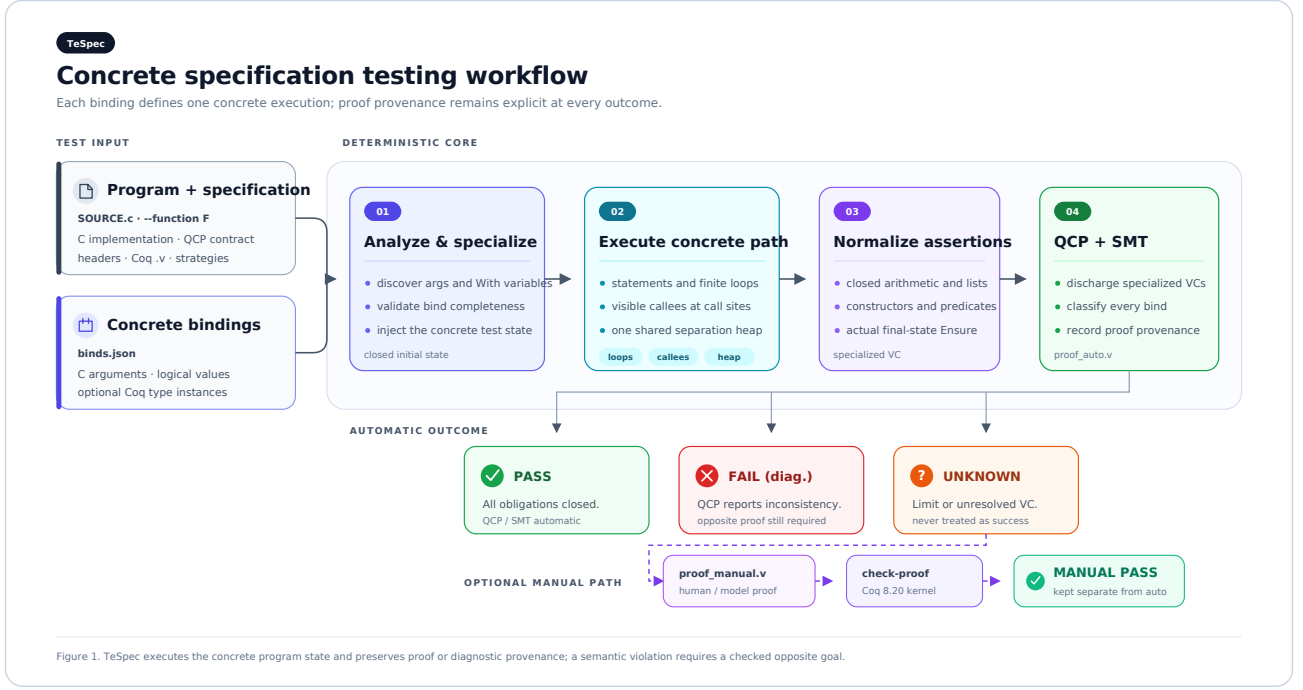


Figure 1: TESPEC workflow. The agent-facing binding and residual-proof steps propose artifacts; parsing, execution, proof generation, and verdict classification remain deterministic.

2.4 The gap between existing approaches

This example locates TESPEC between two established workflows. A Bennet-style workflow can generate and execute rich C heaps when the complete oracle is expressed in its deliberately executable CN fragment. A COINS-style workflow can instantiate a declarative proposition on a trusted input/output pair, but its cases do not include a C execution with ownership, aliasing, globals, and old/new heaps [Yang et al., 2026]. Universal QCP verification preserves all of this state, but asks for the loop invariant and proof needed for every legal input.

TESPEC instead follows one finite C execution while retaining QCP’s logical heap and proof backend. It neither claims Bennet’s random generation throughput nor COINS’s language-independent value model. Its intended use is to test an existing proof-oriented C contract on selected heap behaviors without first rewriting that contract into a separate executable property.

3 Tool Overview

TESPEC accepts:

QCP contract + C implementation + one or more binds.

A bind record describes:

- the target function;
- concrete scalar and pointer arguments;
- arrays, structures, globals, and allocated heap blocks reachable from pointer roots;
- values for `With` variables; and
- execution limits and task-local logical dependencies.

The interface separates semantic evidence from execution diagnostics. Under the strict evidence policy below, a selected contract instance is satisfied, violated, or unresolved. Parsing and dependency errors, incomplete heap descriptions, exhausted execution bounds, safety failures, and unproved QCP inconsistencies are reported separately and are not silently promoted to semantic verdicts.

3.1 A lightweight case model

We use a small amount of notation to make the engineering contract precise. A concrete behavior is:

$$b = (a, H_0, r, H_1),$$

where a contains C arguments, H_0 and H_1 are finite typed heaps, and r is the return value. A bind η assigns values to the **With** variables of the selected QCP specification.

After substitution, a precondition or postcondition becomes a closed proposition:

$$\phi = \text{instantiate}(S, \eta, a, H_0, r, H_1).$$

The testing workflow uses three outcomes:

SATISFIED	a checked proof of the expected proposition is available,
VIOLATED	a checked proof of its opposite is available,
UNKNOWN	neither direction was established within the budget.

This is an operational evidence policy, not a new program logic. In particular, failure of SMT or symbolic execution is not by itself evidence that a specification is false.

Each bind selects one indexed contract instance. This is analogous to a parameterized unit/property test: the result does not claim anything about unseen bindings. Comparative specification evaluation should reuse the same frozen heap cases and binding schema across candidates; interactive use may allow the candidate precondition to materialize its own heap, but such results must not be compared as if they used identical inputs.

4 Approach

4.1 Step 1: discover the testing interface

The analyzer parses the C signature and QCP annotations, then reports:

- ordinary C parameters and their scalar/pointer types;
- value-level and type-level **With** declarations;
- referenced globals and spatial predicates;
- task-local **Import Coq**, **Extern Coq**, and strategy dependencies; and
- heap regions that the precondition can use to initialize execution.

The generated bind template is intentionally typed rather than program specific. It supports integers, machine scalars, nested lists, constructor terms, arrays, closed structures, pointer roots, and a raw QCP/Rocq term escape hatch for application-defined types. The same representation composes: an array cell may contain a structure, a structure may contain list roots, and a list node may contain arrays or closed substructures.

4.2 Step 2: construct and freeze the initial state

In interactive mode, the tool combines the bind with the spatial **Require** to allocate fresh blocks and initialize cells. Pointers are represented internally as block-offset pairs, preserving aliases without depending on process-specific addresses.

The planned publication mode exports the resulting initial state as a case record and replays it independently of the candidate specification. This avoids a candidate-controlled-input problem: two specifications must not receive different list lengths merely because their preconditions materialize different heaps.

All memory required by execution must be concrete. A symbolic frame may be retained for untouched regions during interactive execution, but a comparative case must also fix memory observable by the candidate contract. The evaluator records which cells were read, written, or only framed to make this distinction auditable.

4.3 Step 3: follow the concrete execution

The modified QCP executor preserves the ordinary spatial reasoning needed for loads and stores, but uses concrete values to choose the path.

Branches. An encountered condition must reduce to a concrete Boolean. If it does not, the case is incomplete and returns **UNKNOWN** rather than exploring an arbitrary branch.

Loops. The executor repeatedly evaluates the guard and body until the concrete guard becomes false. **break**, **continue**, and **return** retain their QCP control-flow meanings. A global iteration limit prevents accidental divergence; exhausting it returns UNKNOWN. No loop invariant is requested because the tool records one finite run rather than proving all iterations.

Visible calls. For a call whose body is available, the executor evaluates actual arguments, creates a fresh local frame, executes the callee, propagates the return value and shared heap, and resumes the caller. It therefore does not need a callee contract or callee ghost bind for that call. Opaque external functions require an explicit task-local model.

Safety. Concrete mode does not skip pointer validity, ownership, bounds, or definedness checks. Knowing a numeric address is not the same as owning a valid cell. Failure to derive the spatial resource is reported as an execution/safety failure, not automatically as a failed terminal specification.

4.4 Step 4: specialize and simplify the contract

At the target function boundary, TESPEC substitutes:

$$\eta, \quad a, \quad H_0, \quad r, \quad H_1$$

into the selected **Require** or **Ensure**. The source assertion is preserved; the tool does not synthesize a separate executable checker.

Generic simplification reduces closed arithmetic, Boolean expressions, constructor matches, list length/indexing, known recursive calls, and concrete spatial atoms. Simplification rules operate on typed AST forms rather than function names or benchmark constants. Unsupported constructs remain in the goal.

4.5 Step 5: discharge and report

The backend follows QCP’s normal artifact split:

- a goal file containing generated VCs;
- automatic proofs produced by spatial strategies and SMT;
- a residual proof file for unresolved goals; and
- a final Rocq module that checks proof coverage.

QCP’s SMT layer is effective for closed linear arithmetic and uninterpreted functions, but arbitrary quantified or domain-specific propositions may remain residual. A proof agent may propose a proof script, but the script is accepted only if Rocq compiles it against the original goal. Agent failure leaves the case unresolved.

5 Binding as a User and Agent Interface

Logical binding is often the least familiar part of the workflow. The C run may show a three-node heap, while the contract expects a **list** Z, a nested constructor, or an application-specific Rocq value. TESPEC separates three activities:

1. **Discovery:** deterministic analysis reports all values that must be supplied and their types.
2. **Proposal:** a user or agent fills the template using the pre-state, source contract, and task-local definitions.
3. **Checking:** the deterministic core parses, type-checks, and applies the binding; a malformed proposal cannot change the verdict.

The accompanying Codex skill documents how to inspect the initial heap, map recursive predicates to logical values, run the tool, and handle residual VCs. The skill is a usability layer rather than a component of the semantic oracle. To prevent output leakage, a binding for a pre-state is saved before examining the final heap and reused for positive and mutant executions from that state.

Human-authored binds remain first-class. Every agent-produced bind is stored as ordinary input data, making the run reproducible without the original conversation or model.

6 Implementation

The prototype is packaged independently under `specTest`. Each case stores its C source, binds, Rocq definitions, QCP strategies, and generated evidence locally; it does not depend on a monolithic external case corpus.

The implementation has four main components:

Analyzer	Parses function signatures, QCP contracts, imports, and logical-variable declarations and emits a bind template.
Case builder	Creates typed heap blocks and globals from the binding or replays a frozen case.
Concrete executor	Extends QCP with finite loop execution, visible-call inlining, concrete branch selection, and resource limits.
Evidence backend	Specializes the boundary assertion, runs generic reductions, invokes QCP/SMT, and packages automatic and residual Rocq proof files.

6.1 Supported program shapes

The present engineering target is deterministic sequential C with:

- primitive scalars and supported opaque 64-bit cells;
- arrays and arrays of closed structures;
- nested closed structures and globals;
- singly and doubly linked lists;
- recursive combinations of arrays, structures, and list nodes;
- finite loops, nested loops, visible calls, and bounded recursion; and
- QCP `Let` predicates and task-local transparent Rocq definitions.

Trees, unrestricted graphs, concurrency, function pointers, `goto`, volatile/device effects, and unsupported floating semantics are outside the current claim. They produce an explicit unsupported or unresolved result unless a trusted case-local model is supplied.

6.2 Prototype and publication-mode distinction

The current command-line workflow is already useful for testing a supplied contract against a self-materialized execution. A comparative specification-evaluation experiment requires additional controls:

- freeze candidate-independent initial heaps;
- use the same bind schema and cases across candidate specifications;
- certify positive and negative labels independently;
- distinguish execution/safety diagnostics from terminal-contract violations; and
- require a checked opposite-polarity goal for a conclusive `VIOLATED` result.

These controls define a strict *evaluation mode*; they are not all implemented by the current diagnostic prototype. In particular, the current CLI can surface a QCP-reported inconsistent obligation, but the paper reserves `VIOLATED` for a separately generated and checked opposite-polarity proposition. Diagnostic-only failures must therefore remain outside semantic rejection counts.

7 Benchmark Design

7.1 A four-class task without supplied test cases

The benchmark evaluates whether an LLM or agent can classify a candidate contract, rather than whether it can answer a preselected instance. A question exposes exactly one implementation I and one QCP contract S . It does *not* provide concrete test inputs, parent implementations, or mutation metadata. Mutation relationships between questions are hidden construction lineage and cannot substitute for evidence about the current pair. A subject that predicts a negative property must discover and submit its own C arguments, initial heap, and logical bindings as a checkable counterexample.

For a legal typed state $x = (a, H_0)$, let $B(I, x)$ be the safely terminating observable behavior, including return value, canonical final heap, modeled globals, and external-call events. Let R_I contain all $(x, B(I, x))$, and let R_S contain all (x, y) admitted by the contract’s **Require** and **Ensure** under logical bindings frozen from the initial state. We define:

$$\begin{aligned}\text{Sound}(S, I) &\triangleq R_S \subseteq R_I, \\ \text{Complete}(S, I) &\triangleq R_I \subseteq R_S.\end{aligned}$$

Thus soundness says that the specification admits no behavior the implementation cannot produce, while completeness says that it omits no defined implementation behavior. The first benchmark version is deterministic and sequential.

label	Sound	Complete	interpretation
CORRECT	yes	yes	equals the implementation behavior relation
SOUNDNESS	yes	no	is too strong and omits some implementation behavior
COMPLETE	no	yes	is too weak and admits behavior the implementation cannot produce
INCOMPARABLE	no	no	both omits implementation behavior and admits non-implementation behavior

The class names **SOUNDNESS** and **COMPLETE** mean sound-only and complete-only in this table. We give the formulas because prior work uses these words in both inclusion directions.

7.2 Counterexample certificates

A soundness counterexample is a checked witness

$$(x, y) \in R_S \setminus R_I.$$

A completeness counterexample is a checked witness

$$(x, B(I, x)) \in R_I \setminus R_S.$$

Both certificates contain concrete C arguments, the complete relevant initial heap, pre-state logical bindings, and checked candidate polarity. A soundness counterexample additionally proves that the admitted behavior is not the implementation behavior; a completeness counterexample archives the implementation trace and the candidate rejection. We exclude undefined, crashing, nonterminating, and unmodeled-I/O executions from semantic evidence.

The benchmark has binary gold values for the two properties, but an evaluated tool may abstain. Failure to find a counterexample never establishes a positive property. Gold **yes** axes therefore require a checked inclusion proof (or an explicitly complete finite-domain certificate), while gold **no** axes require the counterexamples above.

7.3 One hundred programs and six questions each

We shortlist 100 difficult annotated target functions and construct six implementation/specification questions per function, for 600 questions. Every base contributes at least one question from each class, three *hard* questions, and three *expert* questions. The two additional class occurrences rotate across bases, yielding exactly 150 questions per class, with 75 in each class/difficulty cell. Hard items require at least one semantic specification transformation and two reasoning dimensions. Expert items require at least two sequential, nonredundant transformations, surface camouflage, and three reasoning dimensions.

The subject pool deliberately combines arrays, strings, closed structures, singly and doubly linked lists, nested control flow, visible calls, native floating-point operations, floating-point models, quantifiers, and custom Rocq

definitions. Selection occurs at the target-function level, but all targets from one source family remain in one data split.

Mutation recipes include equivalent fold/unfold and logical rewrites; weakenings that omit return, heap, ordering, quantifier, call-event, or link-consistency constraints; strengthenings that overconstrain an input, return, heap cell, floating result, or list relation; and mixed mutations for the incomparable class. Implementation mutations cover branches, loop bounds, indices, writes, fields, calls, floating operators, and list links. An operator name does not determine a label: redundant edits and equivalent mutants are discarded after semantic auditing.

7.4 Gold construction, difficulty, and leakage control

For implementation relation R_I and candidate relation R_S , a correct item stores proofs of both inclusions. A sound-only item stores $R_S \subseteq R_I$ and an implementation-only witness. A complete-only item stores $R_I \subseteq R_S$ and a specification-only witness. An incomparable item stores both witnesses. Each item also archives source and dependency hashes, canonical heaps and traces, selected QCP specification, binding-adequacy evidence, normalized propositions, proof polarity, and checker version.

Difficulty scores reject obviously shallow construction plans but are not the authoritative difficulty criterion. Each materialized question receives three independent runs of the frozen OpenHands plus `openai/gpt-5-nano` generic-agent baseline. We retain questions solved in at most one run and replace questions solved in at least two runs with a fresh question of the same class. Infrastructure failures remain unresolved rather than counting as model errors. A scored trajectory must also attest that the agent inspected both public semantic inputs; a label emitted without reading either input is unresolved. Release additionally requires this Nano gate, a difficulty-gate certificate, balanced surface-statistics audit, label-blind mutation review, and, for expert items, a composed-mutation nonredundancy certificate.

The train/development/test split is grouped by project, source family, and predicate family; all six questions for a base program stay together. Mutation and label names are removed from the model-visible package. We balance edit size and contract length across classes and hold out mutation families to test whether a model learned semantics rather than the generator’s surface pattern.

7.5 Model and tool comparison

We use a paired protocol for each model:

1. direct LLM classification from the source package;
2. a generic coding agent with ordinary repository and shell tools; and
3. the same agent and budget with the TESPEC skill and CLI available.

The third condition may use TESPEC to analyze bindings, execute the implementation, replay canonical heaps, and check both candidate proposition polarities. The tool does not receive the hidden class and does not emit the four-way answer. We report four-class accuracy and macro-F1, balanced accuracy on each Boolean axis, and *certificate success*, which requires both the correct label and valid counterexamples for every predicted-false axis. Cost, time, calls, abstention, proof provenance, and feature-stratified results are secondary metrics. Confidence intervals use a base-program-clustered bootstrap because six questions from one program are not independent observations.

7.6 Empirical results

The empirical results are intentionally left blank in this draft. They will be populated only after all 600 questions pass the frozen gold audit; the static shortlist and internal regression runs are not reported as benchmark results.

8 Engineering Assurances

TESPEC does not introduce a new proof calculus. Its evidence chain reuses QCP’s generated-goal, automatic-proof, residual-proof, and final-check module architecture. The Rocq check establishes that each listed goal has a proof; it does not validate a buggy frontend or an omitted VC. We therefore adopt the following engineering controls:

- archive the exact source, bind, heap, generated goal, proof files, tool versions, and hashes for every result;
- prohibit `Admitted`, candidate-supplied axioms, and proof escape hatches;
- compile the final proof-check module for every proof-backed verdict;

Approach	Cases	Native C heap	Test oracle
QuickCheck	/	Generated values	No mutable C ownership
QuickChick			
Fulminate	Supplied executions	Yes	Compiled runtime checks for CN
Bennet	Generated heaps	Yes	Generated CN runtime checks
VERINA / COINS	Fixed positive/negative values	No native C heap transition	Tests and/or proof-checked proposition
TESPEC	Supplied typed heaps and binds	Yes	Specialized QCP/Rocq proof obligation

Table 1: Design-space comparison with representative related systems.

- never classify prover failure as semantic rejection;
- run differential checks between native execution and concrete QCP execution on the supported C subset;
- test generic reductions with name- and constant-renamed programs; and
- maintain held-out projects and predicate families to detect case-specific patches.

These controls support reproducibility and reduce false conclusions, but they are not presented as a mechanized end-to-end soundness theorem for the modified executor.

9 Threats to Validity

Construct validity. Passing finite cases measures agreement with those cases, not universal specification correctness. Mutation score depends on the chosen fault model, and automatically generated mutants may be equivalent. We mitigate these issues by separating positive/negative pre/post results, reviewing mutation operators, and reporting uncertified cases.

Internal validity. Incorrect binds, heap replay, reference contracts, or case labels could bias the result. We store all intermediate artifacts, reuse frozen inputs across candidates, validate binds independently, and require proof-backed labels for the strict experiment.

External validity. QCP and QCIP/CAV subjects may not represent all systems C. The current scope excludes concurrency, graphs, device memory, and several C features. We report results by feature and use held-out projects instead of extrapolating from one corpus.

Conclusion validity. Proof-search timeouts can make a true instance appear unresolved. We keep UNKNOWN separate from rejection, report lower and upper resolution bounds, repeat stochastic agent conditions, and use paired analyses for ablations.

Agent and data leakage. An LLM may have seen public examples or infer a bind from output information. We use held-out transformations and projects, prevent the binding step from seeing terminal states, archive prompts, and report human-only and agent-assisted settings separately.

10 Related Work

Table 1 summarizes the closest design points. The categories describe the default workflow of each system; they are not claims that a framework could never be extended.

10.1 Property-based testing

QuickCheck popularized testing executable properties over generated inputs [Claessen and Hughes, 2000]. Subsequent PBT systems improved generation, shrinking, and integration with proof assistants. QuickChick provides a foundational PBT framework for Coq developments [Paraskevopoulou et al., 2015]; FuzzChick combines property-based generation with coverage-guided mutation to reach behavior hidden behind sparse semantic preconditions [Lampropoulos et al., 2019]. These systems treat the property as an executable decision procedure. In contrast, TESPEC treats the test oracle as a closed proof obligation, which accepts proof-oriented predicates for which no executable checker is supplied, but is necessarily more expensive and may return a residual goal.

PBT research also highlights that generating valid structured inputs is often harder than running the property. TESPEC currently consumes explicit typed bindings rather than claiming a complete random generator. Its bind analyzer and optional agent address authoring effort, while input generation and shrinking remain complementary future work.

10.2 Testing heap specifications

Runtime checking and test generation for heap-manipulating programs are the closest technical relatives. SLICK explored runtime checking of separation-logic invariants [Nguyen et al., 2008]. CSF uses separation-logic preconditions in concolic test generation [Pham et al., 2019]. Fulminate compiles CN specifications to instrumented C and checks ownership transfer at runtime [Banerjee et al., 2025]. Bennet builds on the same executable CN fragment to synthesize backtracking generators for complex heap inputs [Aamer and Pierce, 2025].

This line of work invalidates any broad claim that PBT cannot handle heaps. The meaningful distinction is expressiveness and oracle construction. Fulminate and Bennet intentionally exploit a restricted presentation of separation logic with an effective computational interpretation. TESPEC targets existing QCP specifications that may contain application-defined transparent Rocq definitions, quantified propositions, and residual proof obligations. It follows one supplied heap case and proves the instantiated assertion instead of compiling all supported assertions into runtime code.

10.3 Formal specification evaluation

FormalBench studies specification inference through consistency and completeness and uses mutation-oriented analysis to expose semantic weaknesses [Le-Cong et al., 2025]. CLEVER evaluates generated Lean specifications against curated references with formal equivalence [Thakur et al., 2025]. VERINA decomposes verifiable-code generation into code, specification, and proof tasks, and evaluates specifications with positive and negative test suites [Ye et al., 2026]. COINS directly proves specialized declarative specifications on concrete value-level examples [Yang et al., 2026]. VeriScale expands test suites with adversarial implementations and then reduces them while preserving discriminative power [Bai et al., 2026]. OSVBench broadens specification generation to operating-system verification tasks [Li et al., 2026]. MutDafny uses implementation mutation to identify weak Dafny specifications and derives language-specific operators from real bug-fix commits [Amaral et al., 2026].

TESPEC adopts the central testing insight of this work: desired behaviors measure over-restriction, while undesired behaviors measure under-specification. Its contribution is an engineering path for applying that idea to mutable C heap transitions and QCP separation-logic contracts. Unlike reference-equivalence evaluation, it does not attempt to prove two contracts universally equivalent. Unlike purely value-level instance evaluation, a case includes typed memory, aliasing, global state, and old/new heap contents.

10.4 Verification and agentic assistance

QCP provides the verification infrastructure reused by TESPEC: symbolic execution, a separation-logic entailment solver, SMT, and residual Rocq proofs [Wu et al., 2026]. Standard QCP targets universal function verification; TESPEC specializes this machinery to selected concrete cases for debugging and evaluation.

Recent work explores agents for separation-logic specification synthesis and program verification [Suresh et al., 2026]. TESPEC uses an agent only as an optional interface for proposing binds and residual proofs. This role separation is methodologically important: agent success affects automation and cost, while deterministic parsers, execution, and proof checking determine the recorded evidence.

11 Discussion

A complement to PBT, not a replacement. When a contract has a faithful executable interpretation, compiled PBT offers orders-of-magnitude more cases and mature shrinking. TESPEC is intended for the remaining assertions: those already written for deductive verification whose logical expressiveness prevents direct execution. A practical workflow can use both, routing executable fragments to runtime testing and difficult closed instances to proof-backed checking.

Testing the specification versus testing the implementation. The same execution can serve two purposes. With a trusted contract, mutants test an implementation. With trusted positive and negative behaviors, the behaviors test a candidate contract. A future evaluation will use the latter view: the implementation generates heap transitions, while the candidate specification is the artifact being assessed. The evaluation section remains unpopulated in this draft.

Why concrete symbolic execution? Native execution alone produces bytes and addresses but not the QCP spatial state needed to instantiate the contract. Universal QCP verification requires invariants and callee contracts that are unnecessary for one test. Concrete symbolic execution occupies the middle: it follows one path while retaining the assertion and heap representation expected by the QCP backend.

Where agents add value. Mapping an arbitrary recursive heap to an arbitrary Rocq type and proving a domain-specific residual are open-ended tasks. Agents are useful proposal mechanisms for both, but the core tool remains usable with human-authored binds and proofs. Keeping agent artifacts explicit also enables controlled experiments rather than treating an LLM as an unobservable judge.

12 Conclusion

TESPEC brings test-style feedback to expressive QCP specifications for heap-manipulating C. Users provide concrete program inputs and typed logical bindings; the tool follows finite loops and visible calls, records the heap transition, specializes the original contract, and invokes QCP/SMT/Rocq to check the resulting instance. The design complements executable PBT systems such as Fulminate and Bennet and extends instance-based specification evaluation from ordinary values to arrays, structures, aliasing, and linked heaps.

The design claim is deliberately engineering-oriented: TESPEC should eventually be judged by the QCP contracts it can exercise, the fraction of instances it resolves automatically, the specification faults its positive and negative cases expose, and the effort required to supply binds. This draft reports none of those measurements. The approach does not turn finite testing into universal verification, but provides a path toward reproducible, proof-backed debugging of contracts that do not come with runtime checkers.

References

- Zain K. Aamer and Benjamin C. Pierce. Bennet: Randomized specification testing for heap-manipulating programs. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3924–3953, 2025. doi: 10.1145/3764115.
- Isabel Amaral, Alexandra Mendes, and José Campos. MutDafny: A mutation-based approach to assess Dafny specifications. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. URL <https://arxiv.org/abs/2511.15403>.
- Yifan Bai, Xiaoyang Liu, Zihao Mou, Guihong Wang, Jian Yu, Shuhan Xie, Yantao Li, Yangyu Zhang, Jingwei Liang, and Tao Luo. VeriScale: Adversarial test-suite scaling for verifiable code generation. *arXiv preprint arXiv:2605.22368*, 2026. URL <https://arxiv.org/abs/2605.22368>.
- Rini Banerjee, Kayvan Memarian, Dhruv Makwana, Christopher Pulte, Neel Krishnaswami, and Peter Sewell. Fulminate: Testing CN separation-logic specifications in C. *Proceedings of the ACM on Programming Languages*, 9(POPL):1260–1292, 2025. doi: 10.1145/3704879.
- Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279, 2000. doi: 10.1145/351240.351266.
- Leonidas Lampropoulos, Michael Hicks, and Benjamin C. Pierce. Coverage guided, property based testing. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA), 2019. doi: 10.1145/3360607.
- Thanh Le-Cong, Bach Le, and Toby Murray. Can LLMs reason about program semantics? a comprehensive evaluation of LLMs on formal specification inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 21991–22014. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1068. URL <https://aclanthology.org/2025.acl-long.1068/>.
- Shangyu Li, Juyong Jiang, Tiancheng Zhao, and Jiasi Shen. OSVBench: Benchmarking LLMs on specification generation tasks for operating system verification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. doi: 10.1609/aaai.v40i37.40437. URL <https://arxiv.org/abs/2504.20964>.
- Huu Hai Nguyen, Viktor Kuncak, and Wei-Ngan Chin. Runtime checking for separation logic. In *Verification, Model Checking, and Abstract Interpretation*, pages 203–217, 2008. doi: 10.1007/978-3-540-78163-9_19.
- Zoe Paraskevopoulou, Catalin Hritcu, Maxime Dénès, Leonidas Lampropoulos, and Benjamin C. Pierce. Foundational property-based testing. In *Interactive Theorem Proving*, pages 325–343, 2015. doi: 10.1007/978-3-319-22102-1_22.

- Long H. Pham, Quang Loc Le, Quoc-Sang Phan, and Jun Sun. Concolic testing heap-manipulating programs. In *Formal Methods – The Next 30 Years*, pages 442–461, 2019. doi: 10.1007/978-3-030-30942-8_27.
- Tarun Suresh, David Korczynski, and Julien Vanegue. Agentic separation logic specification synthesis. *arXiv preprint arXiv:2605.27531*, 2026. URL <https://arxiv.org/abs/2605.27531>.
- Amitayush Thakur, Jasper C. H. Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. CLEVER: A curated benchmark for formally verified code generation. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL <https://arxiv.org/abs/2505.13938>.
- Xiwei Wu, Yueyang Feng, Xiaoyang Lu, Tianchuan Lin, Kan Liu, Zhiyi Wang, Shushu Wu, Lihan Xie, Chengxi Yang, Hongyi Zhong, Zihan Zhang, Juanru Li, Naijun Zhan, Zhenjiang Hu, and Qinxiang Cao. QCP: A practical separation logic-based C program verification tool. *arXiv preprint arXiv:2505.12878*, 2026. URL <https://arxiv.org/abs/2505.12878>.
- Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiyi Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=41zGzIEpQd>.
- Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking verifiable code generation. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2505.23135>.